

IsPilot API

This service exposes IsPilot through a REST API for enterprise channels such as Copilot Studio, Microsoft Teams, and custom applications.

Production Status: ☒ Live at <https://ispilot-api-46y2f3tyja-uc.a.run.app>

Primary Consumer: Teams Bridge ([teams-ispilot-bridge](#) Cloud Run service)

Architecture

The API communicates with Google Vertex AI Reasoning Engine (ID: 5375474415045705728) to orchestrate multi-agent conversations. Sessions are managed through Cloud Firestore with automatic fallback to in-memory storage.

```
Teams Request
  ↓
Teams Bridge (Cloud Run - Public)
  ↓
[Generates Identity Token]
  ↓
IsPilot API (Cloud Run - Private)
  ↓
Vertex AI Reasoning Engine
  ↓
Agent Orchestration
  ↓
BigQuery Analytics & Results
```

Security Model

- **Teams Bridge:** Accepts unauthenticated requests from Teams (public), but authenticates to IsPilot API using service account credentials
- **IsPilot API:** Validates identity tokens from Teams Bridge before processing requests (private)
- **Service Account:** Both services run as [sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com](#)
- **Authentication:** Identity token (JWT) generated by Teams Bridge, validated by IsPilot API

For detailed IAM setup, see [docs/PERMISSIONS_AND_IAM.md](#).

Quick Start: Local Development

Setup

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
```

```

pip install -r requirements.txt

# Set environment variables
export GOOGLE_CLOUD_PROJECT="corp-stro-salesinventory-prod"
export GOOGLE_CLOUD_LOCATION="us-central1"
export VERTEX_PROJECT_ID="corp-stro-salesinventory-prod"
export VERTEX_LOCATION="us-central1"
export VERTEX_ENGINE_ID="5375474415045705728"
export FIRESTORE_COLLECTION="user_sessions"
export SESSION_TIMEOUT_HOURS="8"

# For local testing with API key
export ISPILOT_API_KEY="your-test-api-key"

# Run the server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8080

```

Note: Application Default Credentials (ADC) will automatically detect and use available credentials. In local development, ensure you're authenticated:

```
gcloud auth application-default login
```

Using run.sh

```

chmod +x run.sh
./run.sh

```

Authentication

Cloud Run (Production)

Uses **Workload Identity** - no credential files required. The service account (`sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com`) is automatically bound to the Cloud Run service.

Client Authentication (calling the API):

```

# Get an identity token using impersonation for the project service account
TOKEN=$(gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
  prod.iam.gserviceaccount.com)

# Use the token in requests
curl -sS -X POST "https://ispilot-api-46y2f3tyja-uc.a.run.app/chat" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \

```

```
--data '{"user_id":"debug-user","message":"Give me a simple answer about inventory","session_id":"5411387247947677696"}'
```

Critical validation rules:

- This is the working production smoke test for this API.
- `--data` must be valid JSON and must be passed as a single shell string.
- Do not send malformed JSON or truncated payloads; they can produce empty engine output even when the service is healthy.
- When continuing a conversation, reuse the same `session_id` from the earlier response.
- If the project is reused as a template, keep the Cloud Run API auth token generation in this API-level README; the actual agent logic belongs in the root agent source.

IAM impersonation note (important)

A common failure mode is trying to generate the token with a personal Google account that does not have permission to impersonate the Cloud Run service account. The current validated path is:

```
# Activate the project service account directly
gcloud auth activate-service-account \
  sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com \
  --project=corp-stro-salesinventory-prod

# Then generate the identity token without impersonation
TOKEN=$(gcloud auth print-identity-token)

curl -sS -X POST "https://ispilot-api-46y2f3tyja-uc.a.run.app/chat" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{"user_id":"debug-user","message":"hola","session_id":null}'
```

If the request fails with `PERMISSION_DENIED` and a message like `iam.serviceAccounts.getAccessToken or Failed to impersonate ...`, the issue is not the app logic; it is missing IAM permission to impersonate or activate the target service account. The required permission is usually `roles/iam.serviceAccountTokenCreator` on the target service account.

Local Development

Use API Key authentication for testing:

```
curl -X POST http://localhost:8080/chat \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your-test-api-key" \
  -d '{
    "user_id": "sebastian",
    "message": "How is Talca Colin performing?"
  }'
```

Environment Variables

Variable	Required	Description	Default
GOOGLE_CLOUD_PROJECT	☒	GCP project ID	corp-stro-salesinventory-prod
GOOGLE_CLOUD_LOCATION	☒	GCP region	us-central1
VERTEX_PROJECT_ID	☒	Vertex AI project ID	Same as GOOGLE_CLOUD_PROJECT
VERTEX_LOCATION	☒	Vertex AI location	us-central1
VERTEX_ENGINE_ID	☒	Reasoning Engine ID	5375474415045705728
FIRESTORE_COLLECTION		Firestore collection for sessions	user_sessions
SESSION_TIMEOUT_HOURS		Session expiration (hours)	8
ISPILOT_API_KEY		API key for local testing	(none)

Endpoints

GET /health

Health check endpoint (no authentication required).

Response:

```
{
  "status": "healthy",
  "timestamp": "2026-08-28T10:30:45.123456"
}
```

POST /chat

Chat with IsPilot agents.

Request:

```
{
  "user_id": "sebastian",
  "message": "How is Talca Colin performing?",
  "session_id": "optional-session-uuid"
}
```

Response:

```
{
  "answer": "Response from IsPilot...",
  "session_id": "session-uuid",
  "request_id": "request-uuid",
  "timestamp": "2026-08-28T10:30:45.123456",
  "status": "ok"
}
```

Error Response:

```
{
  "error_code": "INVALID_API_KEY",
  "error_message": "Authentication failed",
  "request_id": "request-uuid"
}
```

Session Management

Automatic Session Handling

1. First request creates a session automatically
2. Subsequent requests reuse the session if valid
3. Sessions expire after `SESSION_TIMEOUT_HOURS` (default: 8 hours)
4. Session data persists in Firestore (production) or in-memory (development)

Explicit Session Control

```
# Reuse a specific session
curl -X POST http://localhost:8080/chat \
  -H "X-API-Key: your-api-key" \
  -d '{
    "user_id": "sebastian",
    "message": "Continue from previous context",
    "session_id": "8324292617089056768"
  }'
```

Session Storage

Production (Cloud Run):

- Primary: Cloud Firestore (`user_sessions` collection)
- Fallback: In-memory (if Firestore unavailable)
- Persists across service restarts

- Enables multi-instance deployments

Development (Local):

- In-memory session store
- Sessions reset on application restart
- Sufficient for testing and development

Deployment

Prerequisites

Required Tools

- Docker (version 20.10+)
- gcloud CLI (version 500+)
- bash or PowerShell
- Git

Required Google Cloud Resources

- **Project:** `corp-stro-salesinventory-prod`
- **Service Account:** `sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com` (shared with Teams Bridge)
- **Artifact Registry:** `ispilot-api` in `us-central1-docker.pkg.dev`
- **Secret Manager:** `ispilot-api-key` secret
- **Firestore Database:** For user session storage
- **Vertex AI:** Reasoning Engine with ID `5375474415045705728`
- **Teams Bridge:** Cloud Run service `teams-ispilot-bridge` (primary consumer)

Required IAM Roles (for IsPilot API)

The service account (`sa-tot-osa`) must have:

```
- roles/aiplatform.user
- roles/datastore.user
- roles/secretmanager.secretAccessor
- roles/logging.logWriter
- roles/monitoring.metricWriter
- roles/run.invoker (to be invoked by Teams Bridge)
- roles/iam.serviceAccountTokenCreator (for Teams Bridge token generation)
```

For detailed IAM configuration instructions, see [docs/PERMISSIONS_AND_IAM.md](#).

One-Time Setup

Step 1: Create Artifact Registry Repository (if needed)

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
REPOSITORY="ispilot-api"

gcloud artifacts repositories create "${REPOSITORY}" \
  --project="${PROJECT_ID}" \
  --location="${REGION}" \
  --repository-format=docker
```

Step 2: Create Secret for API Key

```
PROJECT_ID="corp-stro-salesinventory-prod"

# Generate a secure random API key
API_KEY=$(openssl rand -hex 16)

# Create the secret
echo -n "${API_KEY}" | gcloud secrets create ispilot-api-key \
  --project="${PROJECT_ID}" \
  --replication-policy="automatic" \
  --data-file=-

# Grant the service account access to the secret
gcloud secrets add-iam-policy-binding ispilot-api-key \
  --project="${PROJECT_ID}" \
  --member="serviceAccount:sa-tot-osa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --role=roles/secretmanager.secretAccessor

echo "Created secret 'ispilot-api-key': ${API_KEY}"
```

Deployment to Cloud Run

Automated Deployment (Recommended)

```
cd /path/to/ispilot-api
chmod +x deploy.sh
./deploy.sh
```

The script will:

1. Build Docker image
2. Configure Docker authentication
3. Push image to Artifact Registry
4. Deploy to Cloud Run with Workload Identity

5. Set all required environment variables
6. Output service URL

Current Production Deployment:

- URL: <https://ispilot-api-46y2f3tyja-uc.a.run.app>
- Service Account: [sa-tot-osa@corp-stro-salesinventory-prod.iam.gserviceaccount.com](#)
- Region: [us-central1](#)
- Status: ☒ Active

Manual Deployment (Advanced)

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="ispilot-api"
REPOSITORY="ispilot-api"

# 1. Build Docker image
docker build -t us-central1-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${SERVICE_NAME}:latest .

# 2. Configure Docker for Artifact Registry
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

# 3. Push image
docker push us-central1-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${SERVICE_NAME}:latest

# 4. Deploy to Cloud Run
gcloud run deploy "${SERVICE_NAME}" \
  --project "${PROJECT_ID}" \
  --region "${REGION}" \
  --image "us-central1-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${SERVICE_NAME}:latest" \
  --platform managed \
  --no-allow-unauthenticated \
  --service-account "sa-tot-osa@${PROJECT_ID}.iam.gserviceaccount.com" \
  --memory 1Gi \
  --cpu 2 \
  --concurrency 100 \
  --timeout 300 \
  --set-env-vars \
    "GOOGLE_CLOUD_PROJECT=${PROJECT_ID}," \
    "GOOGLE_CLOUD_LOCATION=${REGION}," \
    "VERTEX_PROJECT_ID=${PROJECT_ID}," \
    "VERTEX_LOCATION=${REGION}," \
    "VERTEX_ENGINE_ID=5375474415045705728," \
    "FIRESTORE_COLLECTION=user_sessions," \
    "SESSION_TIMEOUT_HOURS=8" \
  --update-secrets \
    "ISPILOT_API_KEY=ispilot-api-key:latest"
```

Verification

Get Service URL

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="ispilot-api"

SERVICE_URL=$(gcloud run services describe "${SERVICE_NAME}" \
  --project "${PROJECT_ID}" \
  --region "${REGION}" \
  --format='value(status.url)')

echo "Service URL: ${SERVICE_URL}"
```

Test Health Endpoint

```
curl "${SERVICE_URL}/health"
# Expected: {"status": "healthy", "timestamp": "2026-08-28T10:30:45.123456Z"}
```

Test Chat Endpoint with Identity Token

```
TOKEN=$(gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
prod.iam.gserviceaccount.com)

curl -X POST "${SERVICE_URL}/chat" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{"user_id":"test-user","message":"How is Talca Colin performing?"}'
```

View Logs

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="ispilot-api"

# Recent logs
gcloud run services logs read "${SERVICE_NAME}" \
  --project "${PROJECT_ID}" \
  --region "${REGION}" \
```

```
--limit 50

# Real-time logs
gcloud run services logs read "${SERVICE_NAME}" \
  --project "${PROJECT_ID}" \
  --region "${REGION}" \
  --limit 10 \
  --follow
```

CI/CD: Cloud Build Automation

The service uses Cloud Build for automated CI/CD. Each push to the main branch triggers a build pipeline.

Trigger Manual Build

```
gcloud builds submit \
  --project=corp-stro-salesinventory-prod \
  --config=cloudbuild.yaml \
  --region=us-central1
```

View Build History

```
gcloud builds list \
  --project=corp-stro-salesinventory-prod \
  --limit=10
```

Rollback: Revert to Previous Deployment

Get Previous Revision

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="ispilot-api"

gcloud run revisions list \
  --project="${PROJECT_ID}" \
  --region="${REGION}" \
  --service="${SERVICE_NAME}"
```

Deploy Previous Revision

```
PROJECT_ID="corp-stro-salesinventory-prod"
REGION="us-central1"
SERVICE_NAME="ispilot-api"
REVISION_ID="ispilot-api-xxxxx"

gcloud run services update-traffic "${SERVICE_NAME}" \
  --project="${PROJECT_ID}" \
  --region="${REGION}" \
  --to-revisions "${REVISION_ID}=100"
```

Teams Bridge Integration

The IsPilot API is designed to be consumed by the **Teams Bridge** service, built with **Microsoft 365 Agents SDK**.

Teams Bridge Architecture

```
Teams User → Azure Bot Service → Teams Bridge (Cloud Run)
                                     ↓
                               [Generates sa-tot-osa identity token]
                                     ↓
                               IsPilot API (this service)
                                     ↓
                               Vertex AI Reasoning Engine
```

What the Teams Bridge Does

1. **Receives** Teams activities from Azure Bot Service (with JWT validation)
2. **Authenticates** to this API using identity token from sa-tot-osa
3. **Routes** user messages to the `/chat` endpoint
4. **Returns** responses formatted for Teams UI

For Teams Bridge Developers

No changes are required to this API. The Teams Bridge should:

- Validate Azure Bot JWT token before accepting requests
- Generate Google Cloud identity token for sa-tot-osa
- Call this API at: <https://ispilot-api-46y2f3tyja-uc.a.run.app/chat>
- Pass `Authorization: Bearer {identity_token}` header
- Send request body: `{"user_id": "...", "message": "...", "session_id": "..."}"`

For complete Teams Bridge implementation details, see:

- [../docs/TEAMS_SDK_INTEGRATION.md](#) — Full setup guide
- [../docs/ARCHITECTURE_RESET_TEAMS_SDK.md](#) — Architecture rationale

- [../teams_bot_bridge/](#) — Implementation code (when available)

Monitoring

View Metrics

```
PROJECT_ID="corp-stro-salesinventory-prod"
SERVICE_NAME="ispilot-api"

# View in Cloud Console
echo "https://console.cloud.google.com/run/detail/us-central1/${SERVICE_NAME}/metrics?project=${PROJECT_ID}"
```

Set Up Alerts

Use Cloud Monitoring to create alerts for:

- High error rates (>5%)
- High latency (>1s)
- Service unavailability

Security: API Key Rotation

```
PROJECT_ID="corp-stro-salesinventory-prod"

# Generate new key
NEW_API_KEY=$(openssl rand -hex 16)

# Create new secret version
echo -n "${NEW_API_KEY}" | gcloud secrets versions add ispilot-api-key \
  --data-file=- \
  --project="${PROJECT_ID}"

# Cloud Run automatically uses the latest version (no redeployment needed)
echo "New API key: ${NEW_API_KEY}"
```

To Cloud Run

```
chmod +x deploy.sh
./deploy.sh
```

The deployment uses **Workload Identity**:

- No credential files stored in the container
- Service account automatically bound to Cloud Run service

- `google.auth.default()` automatically detects and uses Workload Identity
- No `GOOGLE_APPLICATION_CREDENTIALS` environment variable needed

This approach is secure and follows Google Cloud best practices.

Troubleshooting

Common Issues

Authentication Failed

- Verify Bearer token is valid: `gcloud auth print-identity-token --audiences <api-url>`
- For API Key auth: check `ISPILOT_API_KEY` environment variable
- Ensure service account has required IAM roles

Session Not Persisting

- Check Firestore is enabled: `gcloud services enable firestore.googleapis.com --project corp-stro-salesinventory-prod`
- Service account needs `roles/datastore.user` permission
- In-memory fallback is active if Firestore unavailable

Vertex AI Connection Failed

- Verify Engine ID is correct: `5375474415045705728`
- Check service account has `roles/aiplatform.user` permission
- Confirm region is `us-central1`

Technical Documentation

For implementation details, see:

- [Permissions & IAM](#) - Service account setup and roles
- [Session Service & Firestore](#) - Session management and fallback
- [Authentication Mechanism](#) - Auth evolution and implementation
- [Work Checkpoint](#) - Track completed work across all sprints

Local Development

Quick Start

```
chmod +x run.sh
./run.sh
```

The `run.sh` script:

1. Creates Python virtual environment
2. Installs dependencies from requirements.txt
3. Loads environment from .env

4. Starts uvicorn with auto-reload

Manual Setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/service-account.json"
export GOOGLE_CLOUD_PROJECT="corp-stro-salesinventory-prod"
export GOOGLE_CLOUD_LOCATION="us-central1"
export VERTEX_PROJECT_ID="corp-stro-salesinventory-prod"
export VERTEX_LOCATION="us-central1"
export VERTEX_ENGINE_ID="5375474415045705728"
export ISPILOT_API_KEY="your-test-api-key"
uvicorn app.main:app --reload --host 0.0.0.0 --port 8080
```

Testing

```
# Run all tests
pytest

# Run with coverage
pytest --cov=app tests/

# Run specific test file
pytest tests/test_health.py

# Run with verbose output
pytest -v
```

Troubleshooting

Authentication Issues

- **"Not authenticated with gcloud"**: Run `gcloud auth login`
- **"Permission denied" errors**: Verify service account has required IAM roles
- **"Secret not found"**: Ensure `ispilot-api-key` secret exists in Secret Manager

Health Check Failures

- Service may need time to start (first deployment can take 30-60 seconds)
- Verify Cloud Run resource limits are sufficient
- Check Cloud Run service logs: `gcloud run services logs read ispilot-api --region us-central1`

Cold Start Performance

- Multi-stage Docker build optimizes image size for faster cold starts
- Consider enabling Cloud Run's min instances if cold starts impact users

Architecture Notes

- The service hides all Vertex-specific request details behind `VertexAgentClient`
- Sessions are managed in Firestore with automatic expiration
- All requests include structured JSON logging with request IDs
- API key validation is enforced via middleware
- Authentication failures are logged for security auditing
- Vertex API calls include automatic retry with exponential backoff